JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Duration

java.lang.Object
    java.time.Duration

All Implemented Interfaces:

Serializable, Comparable<Duration>, TemporalAmount

```
public final class Duration
extends Object
implements TemporalAmount, Comparable<Duration>, Serializable
```

A time-based amount of time, such as '34.5 seconds'.

This class models a quantity or amount of time in terms of seconds and nanoseconds. It can be accessed using other duration-based units, such as minutes and hours. In addition, the DAYS unit can be used and is treated as exactly equal to 24 hours, thus ignoring daylight savings effects. See Period for the date-based equivalent to this class.

A physical duration could be of infinite length. For practicality, the duration is stored with constraints similar to Instant. The duration uses nanosecond resolution with a maximum value of the seconds that can be held in a long. This is greater than the current estimated age of the universe.

The range of a duration requires the storage of a number larger than a long. To achieve this, the class stores a long representing seconds and an int representing nanosecond-of-second, which will always be between 0 and 999,999,999. The model is of a directed duration, meaning that the duration may be negative.

The duration is measured in "seconds", but these are not necessarily identical to the scientific "SI second" definition based on atomic clocks. This difference only impacts durations measured near a leap-second and should not affect most applications. See Instant for a discussion as to the meaning of the second and time-scales.

This is a value-based class; programmers should treat instances that are equal as interchangeable and should not use instances for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail. The equals method should be used for comparisons.

Implementation Requirements:

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
static final Duration	MAX	The maximum supported Duration, which is Long.MAX_VALUE seconds and 999,999,999 nanoseconds.
static final Duration	MIN	The minimum supported Duration, which is Long.MIN_VALUE seconds.
static final Duration	ZERO	Constant for a duration of zero.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
Duration	abs()	Returns a copy of this duration with a positive length.
Temporal	addTo (Temporal temporal)	Adds this duration to the specified temporal object.
static Duration	between (Temporal startInclusive Temporal endExclusive)	Obtains a Duration representing the duration between two temporal objects.
int	compareTo (Duration otherDuration)	Compares this duration to the specified Duration.

Duration	**dividedBy**(long divisor)	Returns a copy of this duration divided by the specified value.
long	**dividedBy**(Duration divisor)	Returns number of whole times a specified Duration occurs within this Duration.
boolean	**equals**(Object other)	Checks if this duration is equal to the specified Duration.
static Duration	**from**(TemporalAmount amount)	Obtains an instance of Duration from a temporal amount.
long	**get**(TemporalUnit unit)	Gets the value of the requested unit.
int	**getNano**()	Gets the number of nanoseconds within the second in this duration.
long	**getSeconds**()	Gets the number of seconds in this duration.
List<TemporalUnit>	**getUnits**()	Gets the set of units supported by this duration.
int	**hashCode**()	A hash code for this duration.
boolean	**isNegative**()	Checks if this duration is negative, excluding zero.
boolean	**isPositive**()	Checks if this duration is positive, excluding zero.
boolean	**isZero**()	Checks if this duration is zero length.
Duration	**minus**(long amountToSubtract, TemporalUnit unit)	Returns a copy of this duration with the specified duration subtracted.

Duration	**minus** (Duration duration)	Returns a copy of this duration with the specified duration subtracted.
Duration	**minusDays** (long daysToSubtract)	Returns a copy of this duration with the specified duration in standard 24 hour days subtracted.
Duration	**minusHours** (long hoursToSubtract)	Returns a copy of this duration with the specified duration in hours subtracted.
Duration	**minusMillis** (long millisToSubtract)	Returns a copy of this duration with the specified duration in milliseconds subtracted.
Duration	**minusMinutes** (long minutesToSubtract)	Returns a copy of this duration with the specified duration in minutes subtracted.
Duration	**minusNanos** (long nanosToSubtract)	Returns a copy of this duration with the specified duration in nanoseconds subtracted.
Duration	**minusSeconds** (long secondsToSubtract)	Returns a copy of this duration with the specified duration in seconds subtracted.
Duration	**multipliedBy** (long multiplicand)	Returns a copy of this duration multiplied by the scalar.
Duration	**negated**()	Returns a copy of this duration with the length negated.
static Duration	**of**(long amount, TemporalUnit unit)	

		Obtains a `Duration` representing an amount in the specified unit.
static `Duration`	**`ofDays`**`(long days)`	Obtains a `Duration` representing a number of standard 24 hour days.
static `Duration`	**`ofHours`**`(long hours)`	Obtains a `Duration` representing a number of standard hours.
static `Duration`	**`ofMillis`**`(long millis)`	Obtains a `Duration` representing a number of milliseconds.
static `Duration`	**`ofMinutes`**`(long minutes)`	Obtains a `Duration` representing a number of standard minutes.
static `Duration`	**`ofNanos`**`(long nanos)`	Obtains a `Duration` representing a number of nanoseconds.
static `Duration`	**`ofSeconds`**`(long seconds)`	Obtains a `Duration` representing a number of seconds.
static `Duration`	**`ofSeconds`**`(long seconds, long nanoAdjustment)`	Obtains a `Duration` representing a number of seconds and an adjustment in nanoseconds.
static `Duration`	**`parse`**`(CharSequence text)`	Obtains a `Duration` from a text string such as `PnDTnHnMn.nS`.
`Duration`	**`plus`**`(long amountToAdd, TemporalUnit unit)`	Returns a copy of this duration with the specified duration added.
`Duration`	**`plus`**`(Duration duration)`	Returns a copy of this duration with the

		specified duration added.
Duration	**plusDays**(long daysToAdd)	Returns a copy of this duration with the specified duration in standard 24 hour days added.
Duration	**plusHours**(long hoursToAdd)	Returns a copy of this duration with the specified duration in hours added.
Duration	**plusMillis**(long millisToAdd)	Returns a copy of this duration with the specified duration in milliseconds added.
Duration	**plusMinutes**(long minutesToAdd)	Returns a copy of this duration with the specified duration in minutes added.
Duration	**plusNanos**(long nanosToAdd)	Returns a copy of this duration with the specified duration in nanoseconds added.
Duration	**plusSeconds**(long secondsToAdd)	Returns a copy of this duration with the specified duration in seconds added.
Temporal	**subtractFrom**(Temporal temporal)	Subtracts this duration from the specified temporal object.
long	**toDays**()	Gets the number of days in this duration.
long	**toDaysPart**()	Extracts the number of days in the duration.
long	**toHours**()	

		Gets the number of hours in this duration.
int	**toHoursPart**()	Extracts the number of hours part in the duration.
long	**toMillis**()	Converts this duration to the total length in milliseconds.
int	**toMillisPart**()	Extracts the number of milliseconds part of the duration.
long	**toMinutes**()	Gets the number of minutes in this duration.
int	**toMinutesPart**()	Extracts the number of minutes part in the duration.
long	**toNanos**()	Converts this duration to the total length in nanoseconds expressed as a `long`.
int	**toNanosPart**()	Get the nanoseconds part within seconds of the duration.
long	**toSeconds**()	Gets the number of seconds in this duration.
int	**toSecondsPart**()	Extracts the number of seconds part in the duration.
String	**toString**()	A string representation of this duration using ISO-8601 seconds based representation, such as `PT8H6M12.345S`.
Duration	**truncatedTo**(TemporalUnit unit)	Returns a copy of this `Duration` truncated to the specified unit.

Duration	**withNanos** (int nanoOfSecond)	Returns a copy of this duration with the specified nano-of-second.
Duration	**withSeconds** (long seconds)	Returns a copy of this duration with the specified amount of seconds.

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Field Details

ZERO

public static final Duration ZERO

Constant for a duration of zero.

MIN

public static final Duration MIN

The minimum supported Duration, which is Long.MIN_VALUE seconds.

API Note:

This constant represents the smallest possible instance of Duration. Since Duration is directed, the smallest possible duration is negative. The constant is intended to be used as a sentinel value or in tests. Care should be taken when performing arithmetic on MIN as there is a high risk that ArithmeticException or DateTimeException will be thrown.

Since:

26

MAX

public static final Duration MAX

The maximum supported Duration, which is Long.MAX_VALUE seconds and 999,999,999 nanoseconds.

API Note:

This constant represents the largest possible instance of Duration. The constant is intended to be used as a sentinel value or in tests. Care should

be taken when performing arithmetic on `MAX` as there is a high risk that `ArithmeticException` or `DateTimeException` will be thrown.

Since:

26

Method Details

ofDays

`public static Duration ofDays(long days)`

Obtains a `Duration` representing a number of standard 24 hour days.

The seconds are calculated based on the standard definition of a day, where each day is 86,400 seconds which implies a 24 hour day. The nanosecond in second field is set to zero.

Parameters:

`days` - the number of days, positive or negative

Returns:

a `Duration`, not null

Throws:

`ArithmeticException` - if the input days exceeds the capacity of `Duration`

ofHours

`public static Duration ofHours(long hours)`

Obtains a `Duration` representing a number of standard hours.

The seconds are calculated based on the standard definition of an hour, where each hour is 3,600 seconds. The nanosecond in second field is set to zero.

Parameters:

`hours` - the number of hours, positive or negative

Returns:

a `Duration`, not null

Throws:

`ArithmeticException` - if the input hours exceeds the capacity of `Duration`

ofMinutes

`public static Duration ofMinutes(long minutes)`

Obtains a `Duration` representing a number of standard minutes.

The seconds are calculated based on the standard definition of a minute, where each minute is 60 seconds. The nanosecond in second field is set to zero.

Parameters:

`minutes` - the number of minutes, positive or negative

Returns:

a `Duration`, not null

Throws:

`ArithmeticException` - if the input minutes exceeds the capacity of `Duration`

ofSeconds

```
public static Duration ofSeconds(long seconds)
```

Obtains a `Duration` representing a number of seconds.

The nanosecond in second field is set to zero.

Parameters:

`seconds` - the number of seconds, positive or negative

Returns:

a `Duration`, not null

ofSeconds

```
public static Duration ofSeconds(long seconds,
                                 long nanoAdjustment)
```

Obtains a `Duration` representing a number of seconds and an adjustment in nanoseconds.

This method allows an arbitrary number of nanoseconds to be passed in. The factory will alter the values of the second and nanosecond in order to ensure that the stored nanosecond is in the range 0 to 999,999,999. For example, the following will result in exactly the same duration:

```
Duration.ofSeconds(3, 1);
Duration.ofSeconds(4, -999_999_999);
Duration.ofSeconds(2, 1000_000_001);
```

Parameters:

`seconds` - the number of seconds, positive or negative

`nanoAdjustment` - the nanosecond adjustment to the number of seconds, positive or negative

Returns:

a `Duration`, not null

Throws:

`ArithmeticException` - if the adjustment causes the seconds to exceed the capacity of `Duration`

ofMillis

```
public static Duration ofMillis(long millis)
```

Obtains a `Duration` representing a number of milliseconds.

The seconds and nanoseconds are extracted from the specified milliseconds.

Parameters:

`millis` - the number of milliseconds, positive or negative

Returns:

a `Duration`, not null

ofNanos

```
public static Duration ofNanos(long nanos)
```

Obtains a `Duration` representing a number of nanoseconds.

The seconds and nanoseconds are extracted from the specified nanoseconds.

Parameters:

`nanos` - the number of nanoseconds, positive or negative

Returns:

a `Duration`, not null

of

```
public static Duration of(long amount,
                          TemporalUnit unit)
```

Obtains a `Duration` representing an amount in the specified unit.

The parameters represent the two parts of a phrase like '6 Hours'. For example:

```
Duration.of(3, SECONDS);
Duration.of(465, HOURS);
```

Only a subset of units are accepted by this method. The unit must either have an exact duration or be `ChronoUnit.DAYS` which is treated as 24 hours. Other units throw an exception.

Parameters:

`amount` - the amount of the duration, measured in terms of the unit, positive or negative

`unit` - the unit that the duration is measured in, must have an exact duration, not null

Returns:

a `Duration`, not null

Throws:

`DateTimeException` - if the unit has an estimated duration

`ArithmeticException` - if a numeric overflow occurs

from

`public static Duration from(TemporalAmount amount)`

Obtains an instance of `Duration` from a temporal amount.

This obtains a duration based on the specified amount. A `TemporalAmount` represents an amount of time, which may be date-based or time-based, which this factory extracts to a duration.

The conversion loops around the set of units from the amount and uses the duration of the unit to calculate the total `Duration`. Only a subset of units are accepted by this method. The unit must either have an exact duration or be `ChronoUnit.DAYS` which is treated as 24 hours. If any other units are found then an exception is thrown.

Parameters:

`amount` - the temporal amount to convert, not null

Returns:

the equivalent duration, not null

Throws:

`DateTimeException` - if unable to convert to a `Duration`

`ArithmeticException` - if numeric overflow occurs

parse

`public static Duration parse(CharSequence text)`

Obtains a `Duration` from a text string such as `PnDTnHnMn.nS`.

This will parse a textual representation of a duration, including the string produced by `toString()`. The formats accepted are based on the ISO-8601 duration format `PnDTnHnMn.nS` with days considered to be exactly 24 hours.

The string starts with an optional sign, denoted by the ASCII negative or positive symbol. If negative, the whole duration is negated. The ASCII letter "P" is next in upper or lower case. There are then four sections, each consisting of a number and a suffix. The sections have suffixes in ASCII of "D", "H", "M" and "S" for days, hours, minutes and seconds, accepted in upper or lower case. The suffixes must occur in order. The ASCII letter "T" must occur before the first occurrence, if any, of an hour, minute or second section. At least one of the four sections must be present, and if "T" is present there must be at least one section after the "T". The number part of each section must consist of one or more ASCII digits. The number may be prefixed by the ASCII negative or positive symbol. The number of days, hours and minutes must parse to a `long`. The number of seconds must parse to a `long` with optional fraction. The decimal point may be either a dot or a comma. The fractional part may have from zero to 9 digits.

The leading plus/minus sign, and negative values for other units are not part of the ISO-8601 standard.

Examples:

```
   "PT20.345S" -- parses as "20.345 seconds"
   "PT15M"     -- parses as "15 minutes" (where a minute is 60 seconds)
   "PT10H"     -- parses as "10 hours" (where an hour is 3,600 seconds)
   "P2D"       -- parses as "2 days" (where a day is 24 hours or 86,400 seconds)
   "P2DT3H4M"  -- parses as "2 days, 3 hours and 4 minutes"
   "PT-6H3M"    -- parses as "-6 hours and +3 minutes"
   "-PT6H3M"    -- parses as "-6 hours and -3 minutes"
   "-PT-6H+3M"  -- parses as "+6 hours and -3 minutes"
```

Parameters:

`text` - the text to parse, not null

Returns:

the parsed duration, not null

Throws:

`DateTimeParseException` - if the text cannot be parsed to a duration

between

```
public static Duration between(Temporal startInclusive,
                               Temporal endExclusive)
```

Obtains a `Duration` representing the duration between two temporal objects.

This calculates the duration between two temporal objects. If the objects are of different types, then the duration is calculated based on the type of the first object. For example, if the first argument is a `LocalTime` then the second argument is converted to a `LocalTime`.

The specified temporal objects must support the `SECONDS` unit. For full accuracy, either the `NANOS` unit or the `NANO_OF_SECOND` field should be supported.

The result of this method can be a negative duration if the end is before the start. To guarantee a positive duration, call `abs()` on the result.

Parameters:

`startInclusive` - the start instant, inclusive, not null

`endExclusive` - the end instant, exclusive, not null

Returns:

a `Duration`, not null

Throws:

`DateTimeException` - if the seconds between the temporals cannot be obtained

`ArithmeticException` - if the calculation exceeds the capacity of `Duration`

get

```
public long get(TemporalUnit unit)
```

Gets the value of the requested unit.

This returns a value for each of the two supported units, `SECONDS` and `NANOS`. All other units throw an exception.

Specified by:

`get` in interface `TemporalAmount`

Parameters:

`unit` - the `TemporalUnit` for which to return the value

Returns:

the long value of the unit

Throws:

`DateTimeException` - if the unit is not supported

`UnsupportedTemporalTypeException` - if the unit is not supported

getUnits

```
public List<TemporalUnit> getUnits()
```

Gets the set of units supported by this duration.

The supported units are `SECONDS`, and `NANOS`. They are returned in the order seconds, nanos.

This set can be used in conjunction with `get(TemporalUnit)` to access the entire state of the duration.

Specified by:

`getUnits` in interface `TemporalAmount`

Returns:

a list containing the seconds and nanos units, not null

isPositive

```
public boolean isPositive()
```

Checks if this duration is positive, excluding zero.

A `Duration` represents a directed distance between two points on the time-line and can therefore be positive, zero or negative. This method checks whether the length is greater than zero.

Returns:

true if this duration has a total length greater than zero

Since:

18

isZero

```
public boolean isZero()
```

Checks if this duration is zero length.

A `Duration` represents a directed distance between two points on the time-line and can therefore be positive, zero or negative. This method checks whether the length is zero.

Returns:

true if this duration has a total length equal to zero

isNegative

```
public boolean isNegative()
```

Checks if this duration is negative, excluding zero.

A `Duration` represents a directed distance between two points on the time-line and can therefore be positive, zero or negative. This method checks whether the length is less than zero.

Returns:

true if this duration has a total length less than zero

getSeconds

```
public long getSeconds()
```

Gets the number of seconds in this duration.

The length of the duration is stored using two fields - seconds and nanoseconds. The nanoseconds part is a value from 0 to 999,999,999 that is an adjustment to the length in seconds. The total duration is defined by calling this method and `getNano()`.

A `Duration` represents a directed distance between two points on the time-line. A negative duration is expressed by the negative sign of the seconds part. A duration of -1 nanosecond is stored as -1 seconds plus 999,999,999 nanoseconds.

Returns:

the whole seconds part of the length of the duration, positive or negative

getNano

```
public int getNano()
```

Gets the number of nanoseconds within the second in this duration.

The length of the duration is stored using two fields - seconds and nanoseconds. The nanoseconds part is a value from 0 to 999,999,999 that is an adjustment to the length in seconds. The total duration is defined by calling this method and `getSeconds()`.

A `Duration` represents a directed distance between two points on the time-line. A negative duration is expressed by the negative sign of the seconds part. A duration of -1 nanosecond is stored as -1 seconds plus 999,999,999 nanoseconds.

Returns:

the nanoseconds within the second part of the length of the duration, from 0 to 999,999,999

withSeconds

`public Duration withSeconds(long seconds)`

Returns a copy of this duration with the specified amount of seconds.

This returns a duration with the specified seconds, retaining the nano-of-second part of this duration.

This instance is immutable and unaffected by this method call.

Parameters:

`seconds` - the seconds to represent, may be negative

Returns:

a `Duration` based on this duration with the requested seconds, not null

withNanos

`public Duration withNanos(int nanoOfSecond)`

Returns a copy of this duration with the specified nano-of-second.

This returns a duration with the specified nano-of-second, retaining the seconds part of this duration.

This instance is immutable and unaffected by this method call.

Parameters:

`nanoOfSecond` - the nano-of-second to represent, from 0 to 999,999,999

Returns:

a `Duration` based on this duration with the requested nano-of-second, not null

Throws:

`DateTimeException` - if the nano-of-second is invalid

plus

`public Duration plus(Duration duration)`

Returns a copy of this duration with the specified duration added.

This instance is immutable and unaffected by this method call.

Parameters:

`duration` - the duration to add, positive or negative, not null

Returns:

a `Duration` based on this duration with the specified duration added, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

plus

```
public Duration plus(long amountToAdd,
                     TemporalUnit unit)
```

Returns a copy of this duration with the specified duration added.

The duration amount is measured in terms of the specified unit. Only a subset of units are accepted by this method. The unit must either have an exact duration or be `ChronoUnit.DAYS` which is treated as 24 hours. Other units throw an exception.

This instance is immutable and unaffected by this method call.

Parameters:

`amountToAdd` - the amount to add, measured in terms of the unit, positive or negative

`unit` - the unit that the amount is measured in, must have an exact duration, not null

Returns:

a `Duration` based on this duration with the specified duration added, not null

Throws:

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

plusDays

```
public Duration plusDays(long daysToAdd)
```

Returns a copy of this duration with the specified duration in standard 24 hour days added.

The number of days is multiplied by 86,400 to obtain the number of seconds to add. This is based on the standard definition of a day as 24 hours.

This instance is immutable and unaffected by this method call.

Parameters:

`daysToAdd` - the days to add, positive or negative

Returns:

a `Duration` based on this duration with the specified days added, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

plusHours

`public Duration plusHours(long hoursToAdd)`

Returns a copy of this duration with the specified duration in hours added.

This instance is immutable and unaffected by this method call.

Parameters:

`hoursToAdd` - the hours to add, positive or negative

Returns:

a `Duration` based on this duration with the specified hours added, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

plusMinutes

`public Duration plusMinutes(long minutesToAdd)`

Returns a copy of this duration with the specified duration in minutes added.

This instance is immutable and unaffected by this method call.

Parameters:

`minutesToAdd` - the minutes to add, positive or negative

Returns:

a `Duration` based on this duration with the specified minutes added, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

plusSeconds

`public Duration plusSeconds(long secondsToAdd)`

Returns a copy of this duration with the specified duration in seconds added.

This instance is immutable and unaffected by this method call.

Parameters:

`secondsToAdd` - the seconds to add, positive or negative

Returns:

a `Duration` based on this duration with the specified seconds added, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

plusMillis

`public Duration plusMillis(long millisToAdd)`

Returns a copy of this duration with the specified duration in milliseconds added.

This instance is immutable and unaffected by this method call.

Parameters:

`millisToAdd` - the milliseconds to add, positive or negative

Returns:

a `Duration` based on this duration with the specified milliseconds added, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

plusNanos

`public Duration plusNanos(long nanosToAdd)`

Returns a copy of this duration with the specified duration in nanoseconds added.

This instance is immutable and unaffected by this method call.

Parameters:

`nanosToAdd` - the nanoseconds to add, positive or negative

Returns:

a `Duration` based on this duration with the specified nanoseconds added, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

minus

```
public Duration minus(Duration duration)
```

Returns a copy of this duration with the specified duration subtracted.

This instance is immutable and unaffected by this method call.

Parameters:

`duration` - the duration to subtract, positive or negative, not null

Returns:

a `Duration` based on this duration with the specified duration subtracted, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

minus

```
public Duration minus(long amountToSubtract,
                      TemporalUnit unit)
```

Returns a copy of this duration with the specified duration subtracted.

The duration amount is measured in terms of the specified unit. Only a subset of units are accepted by this method. The unit must either have an exact duration or be `ChronoUnit.DAYS` which is treated as 24 hours. Other units throw an exception.

This instance is immutable and unaffected by this method call.

Parameters:

`amountToSubtract` - the amount to subtract, measured in terms of the unit, positive or negative

`unit` - the unit that the amount is measured in, must have an exact duration, not null

Returns:

a `Duration` based on this duration with the specified duration subtracted, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

minusDays

```
public Duration minusDays(long daysToSubtract)
```

Returns a copy of this duration with the specified duration in standard 24 hour days subtracted.

The number of days is multiplied by 86,400 to obtain the number of seconds to subtract. This is based on the standard definition of a day as 24 hours.

This instance is immutable and unaffected by this method call.

Parameters:

`daysToSubtract` - the days to subtract, positive or negative

Returns:

a `Duration` based on this duration with the specified days subtracted, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

minusHours

```
public Duration minusHours(long hoursToSubtract)
```

Returns a copy of this duration with the specified duration in hours subtracted.

The number of hours is multiplied by 3,600 to obtain the number of seconds to subtract.

This instance is immutable and unaffected by this method call.

Parameters:

`hoursToSubtract` - the hours to subtract, positive or negative

Returns:

a `Duration` based on this duration with the specified hours subtracted, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

minusMinutes

```
public Duration minusMinutes(long minutesToSubtract)
```

Returns a copy of this duration with the specified duration in minutes subtracted.

The number of minutes is multiplied by 60 to obtain the number of seconds to subtract.

This instance is immutable and unaffected by this method call.

Parameters:

`minutesToSubtract` - the minutes to subtract, positive or negative

Returns:

a `Duration` based on this duration with the specified minutes subtracted, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

minusSeconds

```
public Duration minusSeconds(long secondsToSubtract)
```

Returns a copy of this duration with the specified duration in seconds subtracted.

This instance is immutable and unaffected by this method call.

Parameters:

`secondsToSubtract` - the seconds to subtract, positive or negative

Returns:

a `Duration` based on this duration with the specified seconds subtracted, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

minusMillis

```
public Duration minusMillis(long millisToSubtract)
```

Returns a copy of this duration with the specified duration in milliseconds subtracted.

This instance is immutable and unaffected by this method call.

Parameters:

`millisToSubtract` - the milliseconds to subtract, positive or negative

Returns:

a `Duration` based on this duration with the specified milliseconds subtracted, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

minusNanos

`public Duration minusNanos(long nanosToSubtract)`

Returns a copy of this duration with the specified duration in nanoseconds subtracted.

This instance is immutable and unaffected by this method call.

Parameters:

`nanosToSubtract` - the nanoseconds to subtract, positive or negative

Returns:

a `Duration` based on this duration with the specified nanoseconds subtracted, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

multipliedBy

`public Duration multipliedBy(long multiplicand)`

Returns a copy of this duration multiplied by the scalar.

This instance is immutable and unaffected by this method call.

Parameters:

`multiplicand` - the value to multiply the duration by, positive or negative

Returns:

a `Duration` based on this duration multiplied by the specified scalar, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

dividedBy

`public Duration dividedBy(long divisor)`

Returns a copy of this duration divided by the specified value.

This instance is immutable and unaffected by this method call.

Parameters:

`divisor` - the value to divide the duration by, positive or negative, not zero

Returns:

a `Duration` based on this duration divided by the specified divisor, not null

Throws:

ArithmeticException - if the divisor is zero or if numeric overflow occurs

dividedBy

```
public long dividedBy(Duration divisor)
```

Returns number of whole times a specified Duration occurs within this Duration.

This instance is immutable and unaffected by this method call.

Parameters:

divisor - the value to divide the duration by, positive or negative, not null

Returns:

number of whole times, rounded toward zero, a specified Duration occurs within this Duration, may be negative

Throws:

ArithmeticException - if the divisor is zero, or if numeric overflow occurs

Since:

9

negated

```
public Duration negated()
```

Returns a copy of this duration with the length negated.

This method swaps the sign of the total length of this duration. For example, PT1.3S will be returned as PT-1.3S.

This instance is immutable and unaffected by this method call.

Returns:

a Duration based on this duration with the amount negated, not null

Throws:

ArithmeticException - if numeric overflow occurs

abs

```
public Duration abs()
```

Returns a copy of this duration with a positive length.

This method returns a positive duration by effectively removing the sign from any negative total length. For example, PT-1.3S will be returned as PT1.3S.

This instance is immutable and unaffected by this method call.

Returns:

a `Duration` based on this duration with an absolute length, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

addTo

`public Temporal addTo(Temporal temporal)`

Adds this duration to the specified temporal object.

This returns a temporal object of the same observable type as the input with this duration added.

In most cases, it is clearer to reverse the calling pattern by using `Temporal.plus(TemporalAmount)`.

```
// these two lines are equivalent, but the second approach is recommended
dateTime = thisDuration.addTo(dateTime);
dateTime = dateTime.plus(thisDuration);
```

The calculation will add the seconds, then nanos. Only non-zero amounts will be added.

This instance is immutable and unaffected by this method call.

Specified by:

`addTo` in interface `TemporalAmount`

Parameters:

`temporal` - the temporal object to adjust, not null

Returns:

an object of the same type with the adjustment made, not null

Throws:

`DateTimeException` - if unable to add

`ArithmeticException` - if numeric overflow occurs

subtractFrom

`public Temporal subtractFrom(Temporal temporal)`

Subtracts this duration from the specified temporal object.

This returns a temporal object of the same observable type as the input with this duration subtracted.

In most cases, it is clearer to reverse the calling pattern by using `Temporal.minus(TemporalAmount)`.

```
// these two lines are equivalent, but the second approach is recommended
dateTime = thisDuration.subtractFrom(dateTime);
dateTime = dateTime.minus(thisDuration);
```

The calculation will subtract the seconds, then nanos. Only non-zero amounts will be added.

This instance is immutable and unaffected by this method call.

Specified by:

`subtractFrom` in interface `TemporalAmount`

Parameters:

`temporal` - the temporal object to adjust, not null

Returns:

an object of the same type with the adjustment made, not null

Throws:

`DateTimeException` - if unable to subtract

`ArithmeticException` - if numeric overflow occurs

toDays

```
public long toDays()
```

Gets the number of days in this duration.

This returns the total number of days in the duration by dividing the number of seconds by 86,400. This is based on the standard definition of a day as 24 hours.

This instance is immutable and unaffected by this method call.

Returns:

the number of days in the duration, may be negative

toHours

```
public long toHours()
```

Gets the number of hours in this duration.

This returns the total number of hours in the duration by dividing the number of seconds by 3,600.

This instance is immutable and unaffected by this method call.

Returns:

the number of hours in the duration, may be negative

toMinutes

```
public long toMinutes()
```

Gets the number of minutes in this duration.

This returns the total number of minutes in the duration by dividing the number of seconds by 60.

This instance is immutable and unaffected by this method call.

Returns:

the number of minutes in the duration, may be negative

toSeconds

```
public long toSeconds()
```

Gets the number of seconds in this duration.

This returns the total number of whole seconds in the duration.

This instance is immutable and unaffected by this method call.

Returns:

the whole seconds part of the length of the duration, positive or negative

Since:

9

toMillis

```
public long toMillis()
```

Converts this duration to the total length in milliseconds.

If this duration is too large to fit in a `long` milliseconds, then an exception is thrown.

If this duration has greater than millisecond precision, then the conversion will drop any excess precision information as though the amount in nanoseconds was subject to integer division by one million.

Returns:

the total length of the duration in milliseconds

Throws:

`ArithmeticException` - if numeric overflow occurs

toNanos

```
public long toNanos()
```

Converts this duration to the total length in nanoseconds expressed as a `long`.

If this duration is too large to fit in a `long` nanoseconds, then an exception is thrown.

Returns:

the total length of the duration in nanoseconds

Throws:

`ArithmeticException` - if numeric overflow occurs

toDaysPart

```
public long toDaysPart()
```

Extracts the number of days in the duration.

This returns the total number of days in the duration by dividing the number of seconds by 86,400. This is based on the standard definition of a day as 24 hours.

This instance is immutable and unaffected by this method call.

API Note:

This method behaves exactly the same way as `toDays()`.

Returns:

the number of days in the duration, may be negative

Since:

9

toHoursPart

```
public int toHoursPart()
```

Extracts the number of hours part in the duration.

This returns the number of remaining hours when dividing `toHours()` by hours in a day. This is based on the standard definition of a day as 24 hours.

This instance is immutable and unaffected by this method call.

Returns:

the number of hours part in the duration, may be negative

Since:

9

toMinutesPart

```
public int toMinutesPart()
```

Extracts the number of minutes part in the duration.

This returns the number of remaining minutes when dividing `toMinutes()` by minutes in an hour. This is based on the standard definition of an hour as 60 minutes.

This instance is immutable and unaffected by this method call.

Returns:

the number of minutes parts in the duration, may be negative

Since:

9

toSecondsPart

```
public int toSecondsPart()
```

Extracts the number of seconds part in the duration.

This returns the remaining seconds when dividing `toSeconds()` by seconds in a minute. This is based on the standard definition of a minute as 60 seconds.

This instance is immutable and unaffected by this method call.

Returns:

the number of seconds parts in the duration, may be negative

Since:

9

toMillisPart

```
public int toMillisPart()
```

Extracts the number of milliseconds part of the duration.

This returns the milliseconds part by dividing the number of nanoseconds by 1,000,000. The length of the duration is stored using two fields - seconds and nanoseconds. The nanoseconds part is a value from 0 to 999,999,999 that is

an adjustment to the length in seconds. The total duration is defined by calling `getNano()` and `getSeconds()`.

This instance is immutable and unaffected by this method call.

Returns:

the number of milliseconds part of the duration.

Since:

9

toNanosPart

```
public int toNanosPart()
```

Get the nanoseconds part within seconds of the duration.

The length of the duration is stored using two fields - seconds and nanoseconds. The nanoseconds part is a value from 0 to 999,999,999 that is an adjustment to the length in seconds. The total duration is defined by calling `getNano()` and `getSeconds()`.

This instance is immutable and unaffected by this method call.

Returns:

the nanoseconds within the second part of the length of the duration, from 0 to 999,999,999

Since:

9

truncatedTo

```
public Duration truncatedTo(TemporalUnit unit)
```

Returns a copy of this `Duration` truncated to the specified unit.

Truncating the duration returns a copy of the original with conceptual fields smaller than the specified unit set to zero. For example, truncating with the `MINUTES` unit will round down towards zero to the nearest minute, setting the seconds and nanoseconds to zero.

The unit must have a duration that divides into the length of a standard day without remainder. This includes all time-based units on `ChronoUnit` and `DAYS`. Other ChronoUnits throw an exception.

This instance is immutable and unaffected by this method call.

Parameters:

`unit` - the unit to truncate to, not null

Returns:

a `Duration` based on this duration with the time truncated, not null

Throws:

`DateTimeException` - if the unit is invalid for truncation

`UnsupportedTemporalTypeException` - if the unit is not supported

Since:

9

compareTo

```
public int compareTo(Duration otherDuration)
```

Compares this duration to the specified `Duration`.

The comparison is based on the total length of the durations. It is "consistent with equals", as defined by `Comparable`.

Specified by:

`compareTo` in interface `Comparable<Duration>`

Parameters:

`otherDuration` - the other duration to compare to, not null

Returns:

the comparator value, that is less than zero if this duration is less than `otherDuration`, zero if they are equal, greater than zero if this duration is greater than `otherDuration`

equals

```
public boolean equals(Object other)
```

Checks if this duration is equal to the specified `Duration`.

The comparison is based on the total length of the durations.

Overrides:

`equals` in class `Object`

Parameters:

`other` - the other duration, null returns false

Returns:

true if the other duration is equal to this one

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

A hash code for this duration.

Overrides:

`hashCode` in class `Object`

Returns:

a suitable hash code

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

toString

```
public String toString()
```

A string representation of this duration using ISO-8601 seconds based representation, such as `PT8H6M12.345S`.

The format of the returned string will be `PTnHnMnS`, where n is the relevant hours, minutes or seconds part of the duration. Any fractional seconds are placed after a decimal point in the seconds section. If a section has a zero value, it is omitted. The hours, minutes and seconds will all have the same sign.

Examples:

```
   "20.345 seconds"                 -- "PT20.345S"
   "15 minutes" (15 * 60 seconds)   -- "PT15M"
   "10 hours" (10 * 3,600 seconds)  -- "PT10H"
   "2 days" (2 * 86,400 seconds)    -- "PT48H"
```

Note
that multiples of 24 hours are not output as days to avoid confusion with `Period`.

Overrides:

`toString` in class `Object`

Returns:

an ISO-8601 representation of this duration, not null

---

contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.

Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.

All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.